

Student Manual

Defender for Containers

Hardening Kubernetes (AKS) and containerized workloads

Cloud Security Envisioning & Strategy Workshop - Workload Protection Module

Defender for Containers AKS hardening model

01 Discover & assess

Inventory clusters, nodes, pods, registries, and images.
Prioritize risk with recommendations and Secure Score.

02 Reduce attack surface

Harden identity, API access, namespaces, network paths,
pod permissions, private endpoints, and egress.

03 Protect build-to-run

Scan images, patch base layers, use trusted registries,
apply admission controls, and gate risky releases.

04 Detect & respond

Use audit logs, Defender sensor telemetry, runtime
alerts,
attack paths, and Defender XDR incident correlation.

Core teaching point

Container security is not only image scanning. A hardened AKS estate needs posture governance, admission-time controls, runtime telemetry, and response integration.

Generated: 2026-06-17

Table of Contents

- 1. How to use this manual
- 2. Module overview and learning outcomes
- 3. Container security mental model
- 4. Defender for Containers overview
- 5. Architecture and telemetry flow
- 6. Lab readiness and prerequisites
- 7. Implementation phase 1 - Discover and assess
- 8. Implementation phase 2 - Reduce attack surface
- 9. Implementation phase 3 - Protect build-to-run
- 10. Implementation phase 4 - Detect and respond
- 11. Implementation examples and templates
- 12. Operational best practices
- 13. Troubleshooting guide
- 14. Student lab walkthrough
- 15. Scenario exercises
- 16. Knowledge check
- 17. Glossary
- 18. References and further learning

Manual scope

This student manual expands the attached CESE323 slide into a practical AKS hardening guide. It is designed for classroom discussion, guided demonstration, hands-on lab preparation, and post-workshop remediation planning.

1. How to use this manual

This manual supports the CESE323 Defender for Containers module. Use it before the session to prepare, during the session to follow the implementation logic, and after the session as a remediation reference for AKS and containerized workloads.

Use case	How to use this manual	Expected output
Pre-work	Review the vocabulary, prerequisites, and Microsoft Defender for Cloud navigation path.	Students know what accounts, permissions, portals, and lab assets are needed.
Instructor-led walkthrough	Follow the four-phase hardening workflow: discover, reduce exposure, protect build-to-run, and respond.	Students can explain how Defender for Containers maps to AKS risk reduction.
Hands-on lab	Use the implementation examples, checklists, and evidence worksheet.	Students can enable/review Defender for Containers and interpret findings.
Operational follow-up	Use the remediation model, RACI suggestions, and troubleshooting guidance.	Teams can convert findings into owner-assigned security work items.

Safety note for live environments

Do not enable paid Defender plans, deploy extensions, modify AKS policies, or change production clusters without approval from the subscription owner and the customer change authority. Use a lab subscription whenever possible.

2. Module overview and learning outcomes

Defender for Containers is positioned as part of the Microsoft Defender for Cloud workload protection and posture-management stack. It helps platform and security teams detect Kubernetes and container risks across asset inventory, workload configuration, image vulnerability exposure, runtime behavior, and security operations workflows.

Learning outcome	Student should be able to...
Explain the scope	Differentiate posture management, image vulnerability assessment, admission-time controls, runtime detection, and XDR response.
Map AKS risks	Identify likely risk surfaces: Kubernetes API, RBAC, namespaces, node access, secrets, image supply chain, network paths, and runtime behavior.
Enable and validate	Describe the portal path to enable the Containers plan and verify plan components and coverage.

Learning outcome	Student should be able to...
Interpret findings	Use recommendations, vulnerability findings, Secure Score impact, and affected resources to prioritize remediation.
Implement hardening	Apply least privilege, namespace isolation, network policy, workload identity, pod security, registry controls, and runtime monitoring.
Respond operationally	Convert alerts and recommendations into an owner-assigned remediation workflow with evidence and retesting.

The module is intentionally practical: the outcome is not only to understand the product. Students should be able to explain what to check, what to enable, what evidence to collect, and how to decide which risk to remediate first.

3. Container security mental model

AKS security is easier to reason about when separated into four planes. Each plane has its own failure modes, controls, evidence, and responsible teams.

Plane	What it contains	Typical risk	Primary controls
Control plane	Kubernetes API server, cluster configuration, RBAC, admission controls, namespaces, secrets access, and policies.	Over-permissive identities, public API exposure, weak auditability, risky admission settings.	Microsoft Entra integration, Azure RBAC/Kubernetes RBAC, private cluster/API restrictions, policy/admission control, audit logs.
Supply chain	Base images, Dockerfiles, registries, build pipelines, image tags, dependencies, and release gates.	Vulnerable images, untrusted registries, mutable tags, secrets in images, unsigned images.	ACR controls, image scanning, trusted registries, rebuild from patched base images, signed images, gated deployment.
Runtime plane	Nodes, pods, containers, service accounts, processes, network connections, volumes, and lateral movement paths.	Privilege escalation, root containers, crypto-mining, malware, abnormal process execution, unrestricted pod-to-pod traffic.	Defender sensor, eBPF telemetry, pod security, read-only file systems, resource limits, network policy, runtime alerts.
Response plane	Defender for Cloud recommendations, Secure Score, attack paths, Defender XDR incidents, owners, SLAs, tickets, and evidence.	Findings are ignored, unowned, duplicated, or fixed without validation.	Triage model, RACI, remediation SLAs, evidence capture, retesting, Defender XDR incident workflow.

Simplified Defender for Containers telemetry flow

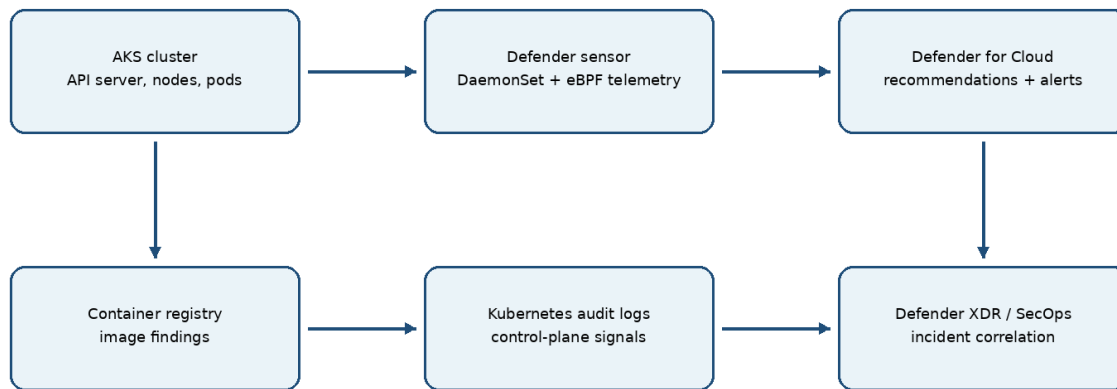


Figure 1. Simplified telemetry and evidence flow for Defender for Containers.

Key distinction

Image scanning answers: “What known vulnerabilities exist in the image?” Runtime protection answers: “What suspicious behavior is happening in the running workload or cluster?” CSPM answers: “Which configuration or exposure increases the probability and blast radius of compromise?”

4. Defender for Containers overview

Microsoft Defender for Containers protects container environments by combining cloud posture management, vulnerability assessment, runtime threat detection, and security operations integration. For AKS, the plan is enabled from Microsoft Defender for Cloud at the subscription/environment level and then validated against specific clusters and components.

Capability area	What students should look for	Example evidence
Asset discovery	Clusters, nodes, workloads, images, registries, and connected Kubernetes environments.	Inventory records, Defender for Cloud resources, Kubernetes recommendations.
Security posture	Misconfigurations, exposed resources, missing policies, identity risks, network exposure, and Secure Score impact.	Recommendations and affected resources with remediation steps.
Vulnerability assessment	Vulnerable registry images and vulnerable images used by running containers.	Vulnerability tab, CVE/package details, affected image digest, fixed version.

Capability area	What students should look for	Example evidence
Runtime detection	Suspicious API activity, abnormal process/network behavior, malware or crypto-mining indicators, escalation attempts.	Defender for Cloud alerts and Defender XDR incident correlation.
Admission/deployment gating	Security rules that prevent risky Kubernetes deployments from entering runtime where gating is enabled.	Security-gating rule, blocked deployment, policy exception record.
Response integration	Alerts, incidents, owners, due dates, and remediation tasks.	Defender XDR incident timeline, work item, evidence screenshot.

Subscription scope matters

When a Defender plan is enabled at subscription scope, protections apply to resources in that subscription. In training, keep scope narrow and cost-controlled. In production, use management group/subscription strategy and written change approval.

5. Architecture and telemetry flow

For AKS, Defender for Containers uses a combination of native Azure control-plane signals, Kubernetes API connectivity, sensor-based telemetry, and registry/runtime vulnerability signals. The attached training slide highlights that runtime telemetry and response integration are as important as image scanning.

Component	Role in the architecture	Operational notes
Kubernetes API connectivity	Used to discover clusters, collect configuration data, and perform posture/risk analysis.	Private clusters and restricted networks must allow the required access patterns.
Kubernetes audit logs	Provide control-plane signals for threat detection and security analysis.	Useful for suspicious API calls, escalation attempts, and abnormal cluster actions.
Defender sensor	Runs as a lightweight DaemonSet on nodes and collects runtime telemetry such as Kubernetes events, process, and network data using eBPF.	Validate sensor health after enabling the plan; ensure outbound HTTPS 443 where required.
Registry access	Enables vulnerability assessment for supported registries and preserves image/repository context.	Required for many registry scanning and gating workflows.
Defender for Cloud backend	Performs analysis, generates recommendations, alerts, vulnerability findings, and Secure	Security teams review findings in Defender for Cloud and incidents in

Component	Role in the architecture	Operational notes
	Score impact.	Defender XDR.

What to validate after enablement

Check that the Containers plan is On, the required plan components are enabled, relevant clusters appear in inventory, recommendations populate, Defender sensor components are healthy where applicable, and test findings/alerts are visible to the right security roles.

6. Lab readiness and prerequisites

The Defender for Containers lab requires more than portal access. Students need a safe lab scope, a supported AKS environment, correct roles, network connectivity, and cost approval. These prerequisites should be validated before class.

Prerequisite	Required for	Validation question
Azure subscription or isolated lab resource group	All hands-on labs.	Can you see the subscription/resource group in the Azure portal?
Microsoft Defender for Cloud enabled	CSPM, recommendations, Secure Score, workload protection plan configuration.	Can you open Defender for Cloud > Environment settings?
AKS cluster	Live Defender for Containers and AKS hardening labs.	Is there a cluster available, supported, and safe to modify?
Supported Kubernetes version	Defender component compatibility and AKS support.	Is the cluster within the supported AKS/Kubernetes version window?
Network egress and private connectivity readiness	Sensor telemetry, API access, registry access, and portal workflows.	Do firewall/proxy/private-cluster controls allow required traffic?
Container registry	Image scanning and supply-chain examples.	Is there a supported registry with non-production images?
Correct roles	Viewing recommendations, enabling plans, assigning/remediating findings.	Do students have Reader/Contributor/Security Admin/Owner only as required?
Cost approval and cleanup plan	AKS resources and Defender paid plans.	Who approved cost and who disables/deletes lab resources after completion?

Student safety boundary

Do not use real customer data, production secrets, HR/legal/financial data, or confidential code in labs. Use sample images and non-production workloads only.

7. Implementation phase 1 - Discover and assess

Discovery is the foundation of container hardening. Teams cannot fix what they do not inventory, and they cannot prioritize what they do not contextualize. Defender for Containers helps identify AKS resources and container risks, while Defender CSPM provides posture context, recommendations, and Secure Score impact.

1. Open Microsoft Defender for Cloud and confirm that the lab subscription is visible.
2. Open Environment settings and check whether the Containers plan is enabled for the target subscription.
3. Review inventory for AKS clusters, registries, images, nodes, and workloads.
4. Filter recommendations for Kubernetes, AKS, containers, registries, and workload identity/RBAC risks.
5. Open a recommendation and capture evidence: affected resource, severity, remediation steps, owner, and business impact.
6. Prioritize findings using exposure, exploitability, business criticality, Secure Score impact, and remediation effort.

Finding category	Example finding	Student interpretation
Identity/RBAC	Cluster admin privileges assigned too broadly.	High blast radius: a compromised user or pipeline identity can modify workloads or secrets.
Network exposure	Publicly reachable service or insufficient network policy.	Exposure risk: a vulnerable pod or service has more reachable attack paths.
Pod security	Privileged container or root execution allowed.	Runtime breakout/escalation risk: workload can interact too broadly with host/node resources.
Image vulnerability	Running image contains critical CVE with fixed package version.	Patch or rebuild image; confirm affected deployment and business owner.
Coverage gap	Defender sensor or required component not healthy.	Detection gap: runtime alerts may be incomplete or delayed.

Example 1 - Baseline AKS and registry discovery

```
# Example Azure CLI discovery commands for lab context
az account show --query "{subscription:name, id:id, tenant:tenantId}" -o table
az aks list -o table
az acr list -o table
```



```
# Optional: set context to a lab AKS cluster
az aks get-credentials -g rg-cese323-lab -n aks-cese323-student --overwrite-existing
kubectl get nodes -o wide
kubectl get ns
kubectl get pods -A
```

8. Implementation phase 2 - Reduce attack surface

Attack-surface reduction changes the environment so that compromise is harder, lateral movement is restricted, and the blast radius of a failure is smaller. In AKS this includes identity, API access, namespace isolation, pod security, network segmentation, private connectivity, and egress control.

Control	Best practice	Implementation pattern
Azure RBAC	Grant the minimum Azure role at the smallest practical scope.	Reader for observation; Contributor on lab resource group only; Security Admin/Owner only for plan enablement where needed.
Kubernetes RBAC	Use namespace-scoped roles instead of broad cluster-admin assignments.	Bind groups to Role/RoleBinding per namespace; reserve ClusterRoleBinding for platform operators.
Microsoft Entra integration	Use Microsoft Entra identities and groups for API server access.	Map Entra groups to Kubernetes roles; avoid shared kubeconfig admin credentials.
Namespace isolation	Separate dev/test/prod, teams, and trust zones.	Use labels, quotas, network policies, and admission rules per namespace.
Network policy	Default deny pod traffic, then explicitly allow required paths.	Allow frontend to backend, backend to database, DNS/logging/monitoring egress.
Pod security	Avoid privileged/root containers and unnecessary Linux capabilities.	Set runAsNonRoot, readOnlyRootFilesystem, seccompProfile, resource limits, and restricted policies.
Workload identity	Avoid storing Azure credentials/secrets in pods.	Use Microsoft Entra Workload ID for pod-to-Azure access such as Key Vault, Storage, or Cosmos DB.
Egress control	Limit outbound destinations from workloads and nodes.	Use firewall/proxy/private endpoints and allowed FQDN/IP rules for required services.

Example 2 - Namespace-scoped reader access

```
# Example namespace and least-privilege Kubernetes RBAC pattern
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: payments-dev
  labels:
    environment: dev
    data-classification: internal
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: payments-dev
  name: deployment-reader
rules:
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bind-deployment-reader
  namespace: payments-dev
subjects:
- kind: Group
  name: <entra-group-object-id>
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: deployment-reader
  apiGroup: rbac.authorization.k8s.io
```

Example 3 - Default deny as a starting point

```
# Example default-deny network policy for a namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: payments-dev
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

Example 4 - Secure pod settings

```
# Example pod security context baseline
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: payments-dev
spec:
  replicas: 2
  selector:
```

```

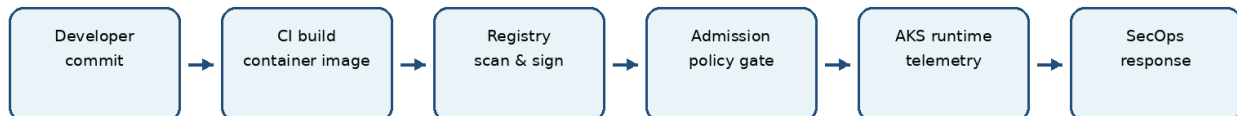
matchLabels:
  app: api
template:
  metadata:
    labels:
      app: api
  spec:
    securityContext:
      seccompProfile:
        type: RuntimeDefault
    containers:
    - name: api
      image: contoso.azurecr.io/payments-api:1.4.3
      securityContext:
        runAsNonRoot: true
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"

```

9. Implementation phase 3 - Protect build-to-run

Build-to-run protection connects DevSecOps and runtime protection. Container security must start before deployment, but it must also validate what is actually running. Defender for Containers can assess vulnerabilities in supported registries and running container images, while deployment gating can prevent risky workloads from being admitted where configured.

Secure build-to-run loop



Feedback loop: findings create work items, patches are rebuilt, and policy gates prevent repeated exposure.

Figure 2. A secure build-to-run loop connects vulnerability findings back to developers and release gates.

Stage	Student action	Best-practice implementation
Dockerfile/base image	Check whether base image is	Use official/minimal images; pin by

Stage	Student action	Best-practice implementation
	maintained, minimal, and pinned.	digest for high-assurance workloads; rebuild regularly.
Build pipeline	Run static checks, secret scanning, and image vulnerability checks.	Fail builds on critical vulnerabilities with available fixes, except documented exceptions.
Registry	Store images in a trusted registry and enable registry vulnerability assessment.	Use ACR permissions, private endpoints where appropriate, immutable tags, retention policy, and registry access controls.
Admission	Block or warn on images that violate policy.	Use gated deployment/security rules and admission controls for privileged pods, untrusted registries, or critical CVEs.
Runtime	Compare running images to registry findings and workload context.	Prioritize images that are both vulnerable and actively running in critical namespaces.
Feedback	Create remediation tickets with image digest, package, fixed version, and owner.	Patch dependency/base image, rebuild, redeploy, and validate that the finding is resolved.

Example 5 - Release gate logic

```
# Example pipeline guardrail pseudo-logic
# 1. Build image
# 2. Push image to trusted registry
# 3. Wait for scan/finding import where available
# 4. Query or review findings
# 5. Block production deployment if:
#   - critical vulnerability has a fix available
#   - image is not from an approved registry
#   - image tag is mutable or unsigned for regulated workload
#   - workload requests privileged execution
# 6. Create exception only with owner, expiry date, and compensating controls
```

Important nuance

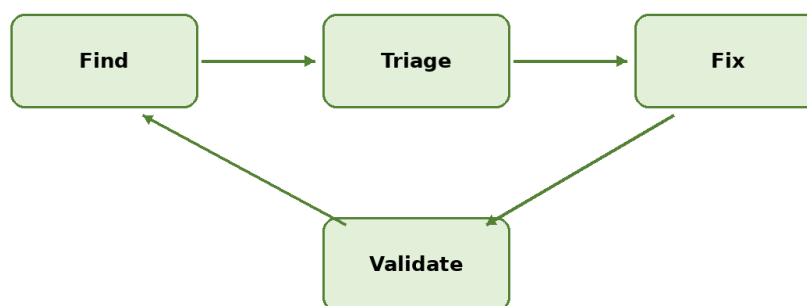
Runtime vulnerability assessment evaluates images used by running containers in supported Kubernetes environments. It does not scan the container runtime layer itself. A clean image scan does not replace runtime monitoring, pod hardening, or network segmentation.

10. Implementation phase 4 - Detect and respond

Detection and response turns telemetry into operational action. Defender for Containers can generate alerts from Kubernetes audit logs and sensor telemetry. These signals should be correlated with Defender XDR incidents, triaged by severity and context, and converted into remediated workload changes.

Signal	Possible meaning	Responder action
Suspicious Kubernetes API activity	Potential unauthorized change, enumeration, credential misuse, or privilege escalation attempt.	Review actor identity, source IP, verb/resource, namespace, recent role assignments, and change history.
Privileged container behavior	Workload may be attempting host access or elevated execution.	Inspect deployment spec, securityContext, admission history, and owning team.
Crypto-mining/malware indicator	Compromised image, vulnerable app, or exposed service.	Isolate workload, capture evidence, rotate secrets, rebuild image from clean base, and redeploy.
Vulnerable running image	Known package vulnerability exists in an actively running image.	Prioritize by severity, exploitability, internet exposure, and business criticality.
Coverage or component health issue	Defender sensor or required connector/component is not working.	Resolve egress, permissions, version, private cluster, or extension issues before relying on coverage.

Operational remediation loop



Risk should become an owner, due date, pull request, deployment, evidence artifact, and closure decision.

Figure 3. Recommended remediation loop for AKS and container security findings.

Example 6 - Alert triage evidence structure

```
# Example triage notes for a container runtime alert
Alert title: Suspicious process execution in Kubernetes container
```

```

Cluster: aks-prod-weu-01
Namespace: payments-prod
Workload: payments-api
Image: contoso.azurecr.io/payments-api@sha256:<digest>
First seen: <timestamp>
Actor/source: <identity or node context>
Business owner: Payments platform team
Immediate action: isolate pod or scale deployment to known-good version
Remediation: rebuild image, patch vulnerable package, rotate secrets
Evidence: Defender XDR incident, pod spec, image digest, deployment history
Closure criteria: alert resolved, vulnerable image no longer running, owner sign-off

```

11. Implementation examples and templates

11.1 Enable Defender for Containers

Portal path: Azure portal > Microsoft Defender for Cloud > Environment settings > select subscription > Defender plans > Containers > On > Settings > enable required components. After enabling, verify that the Containers plan is set to On and required components are enabled for the scope.

Example 7 - CLI readiness checks

```

# Example Azure CLI setup check - exact commands can change by API version.
# Use portal for class if students do not have CLI prepared.
az account set --subscription <subscription-id>
az provider register --namespace Microsoft.Security
az provider register --namespace Microsoft.ContainerService

# Confirm AKS and Defender resource context
az aks show -g rg-cese323-lab -n aks-cese323-student --query provisioningState
az security pricing list -o table

```

11.2 Verify Kubernetes context and Defender components

Example 8 - Sensor/component validation commands

```

# Confirm the current Kubernetes context
kubectl config current-context
kubectl get nodes
kubectl get ns

# Look for Defender-related components where deployed
kubectl get pods -n kube-system | grep -i defender
kubectl get daemonset -n kube-system | grep -i defender
kubectl describe pods -n kube-system <defender-pod-name>

# Capture safe evidence for class
kubectl get pods -A -o wide > evidence-pods.txt
kubectl get events -A --sort-by=.lastTimestamp > evidence-events.txt

```

11.3 Workload identity design pattern

Workload identity allows pods to access Microsoft Entra-protected Azure resources without storing long-lived credentials in Kubernetes secrets. For example, a pod can access Key Vault through a federated service account and a managed identity instead of embedding client secrets in environment variables.

Example 9 - Workload identity implementation checklist

```
# High-level steps for Microsoft Entra Workload ID pattern
# 1. Enable workload identity and OIDC issuer on the AKS cluster.
# 2. Create or choose a user-assigned managed identity.
# 3. Grant the identity least-privilege access to the target Azure resource.
# 4. Create a Kubernetes service account with the required annotations.
# 5. Create federated identity credentials linking service account to identity.
# 6. Configure the pod to use the service account.
# 7. Validate access and remove legacy secrets.
```

11.4 Recommendation triage worksheet

Field	Student input example
Recommendation title	Containers running in Azure should have vulnerability findings resolved.
Affected resource	aks-cese323-student / payments-dev deployment / image digest.
Severity and risk	Critical CVE in internet-facing API image; exploitability public; fix available.
Owner	Payments application team; platform team supports deployment.
Remediation	Update base image, rebuild, scan, redeploy, validate no vulnerable digest running.
Compensating control	Temporary WAF rule and network policy restriction until patch deployed.
Due date	24-72 hours for critical internet-exposed workload.
Evidence for closure	New image digest, scan result, deployment version, Defender recommendation resolved.

11.5 Before/after hardening example

Area	Before	After
Azure RBAC	All engineers have Contributor at subscription scope.	Engineers have Contributor only on lab/dev resource group; Security Admin limited to security leads.
Kubernetes RBAC	Developers use cluster-admin	Developers use Microsoft Entra groups

Area	Before	After
	kubeconfig.	mapped to namespace-scoped roles.
Network	All pods can communicate with all pods.	Default deny policies plus explicit paths for frontend, API, database, DNS, logs, and monitoring.
Pod security	Containers run as root and allow privilege escalation.	runAsNonRoot, seccomp RuntimeDefault, read-only root file system, dropped capabilities.
Secrets	Secrets stored in environment variables and YAML.	Workload identity plus Key Vault access; no long-lived credentials in manifests.
Supply chain	Image uses latest tag from public registry.	Pinned image from trusted registry; vulnerabilities tracked and releases gated.
Response	Findings discussed but not owned.	Findings assigned to owners with SLAs, evidence, retest, and closure criteria.

12. Operational best practices

Practice	Why it matters	Implementation detail
Use least privilege everywhere	Identity compromise is a common path to cluster compromise.	Apply least privilege in Azure RBAC, Kubernetes RBAC, CI/CD service connections, and registry access.
Separate ownership	Platform and app teams fix different issues.	Map recommendations to app owner, platform owner, security owner, and change owner.
Use risk-based prioritization	Security teams cannot remediate every finding at once.	Prioritize internet exposure, privileged workloads, active exploits, critical data paths, and running vulnerable images.
Require remediation evidence	Closure without evidence creates audit risk.	Capture before/after recommendation state, deployment version, image digest, and approval where needed.
Define exception handling	Some critical fixes need time.	Require business owner, expiry date, compensating controls, and re-review date.
Clean up training resources	AKS and Defender paid plans can generate cost.	Delete clusters and unused resources; disable paid plans where no longer

Practice	Why it matters	Implementation detail
		required.
Monitor coverage	Coverage gaps can create false confidence.	Review plan status, component health, sensor status, and unsupported clusters regularly.

Role	Responsibilities in AKS container hardening
Security architect	Define standards, risk acceptance rules, Defender plan strategy, and compliance mapping.
Cloud/platform engineer	Enable plan components, configure AKS controls, validate network/identity/policy, and support remediation.
Application owner	Fix vulnerable images, update manifests, test changes, and accept/reject exceptions.
SecOps analyst	Investigate alerts/incidents, correlate signals, escalate high-risk findings, and validate response.
Change manager	Approve live-environment changes, ensure rollback plan, and track audit evidence.
Student/lab participant	Observe, collect evidence, complete worksheets, and avoid production/sensitive data actions.

13. Troubleshooting guide

Symptom	Likely cause	Action
Cannot enable Containers plan	Insufficient Azure permissions or wrong subscription.	Confirm target subscription, role assignment, and Security Admin/Contributor/Owner requirements.
No AKS recommendations visible	Plan not enabled long enough, wrong scope, no cluster, unsupported cluster, or inventory delay.	Check Environment settings, inventory, cluster support, and wait for assessment to populate.
Defender sensor not healthy	Network egress, private cluster, proxy, policy, or extension issue.	Review architecture/network requirements and pod status in kube-system.
Vulnerability findings missing	Registry access disabled, unsupported registry/image, scan not complete, image not recently pushed/pulled/running.	Confirm Registry Access, supported registry, image activity, and scanning timing.

Symptom	Likely cause	Action
kubectl access denied	Missing Kubernetes RBAC or wrong context.	Check current context, user identity, role bindings, and namespace scope.
Network policy breaks app	Default deny applied before allow rules.	Add explicit DNS, logging, monitoring, and service-to-service allow rules; test in dev first.
Unexpected cost	AKS cluster left running or paid Defender plans left enabled.	Use budgets/alerts, cleanup checklist, and plan disablement where appropriate.
Production change risk	Live environment used without change approval.	Stop changes, document current state, involve owner/change authority, and follow rollback plan.

14. Student lab walkthrough

This is a structured student walkthrough for a lab or guided demo. The instructor can run it portal-first, with optional CLI commands for technical audiences.

Step	Task	Evidence to capture
1	Confirm lab scope: subscription, resource group, AKS cluster, registry, and permissions.	Screenshot or notes showing target scope and role.
2	Open Microsoft Defender for Cloud > Environment settings.	Screenshot of subscription and Defender plan status.
3	Enable or review Containers plan and settings.	Screenshot of Containers On and component settings.
4	Open Defender for Cloud Recommendations and filter for Kubernetes/containers/AKS.	One recommendation with affected resources and remediation steps.
5	Review vulnerability findings for container images where available.	CVE/package/fixed version and affected image digest.
6	Inspect an AKS workload configuration.	Namespace, deployment, security context, service account, and network policy notes.
7	Propose remediation.	Owner, action, priority, due date, and evidence plan.
8	Validate detection/visibility.	Sensor/component health and alert/incident visibility where

Step	Task	Evidence to capture
		available.
9	Cleanup.	Deleted lab resources or documented plan to retain with approval.

Evidence worksheet field	Student response
Subscription/resource group	
AKS cluster name	
Containers plan status	
Recommendation reviewed	
Affected resource	
Risk explanation	
Proposed remediation	
Owner/team	
Priority and due date	
Validation evidence	
Cleanup status	

15. Scenario exercises

Scenario A - Critical image vulnerability in production namespace

Context: A critical CVE with a fixed version is detected in an image running in the payments-prod namespace. The application is public-facing and the team is one week from a release freeze.

Student task: Decide whether to hotfix, temporarily restrict exposure, or request an exception. Identify evidence needed for closure.

Decision area	Student notes
Highest risk	
Immediate action	

Decision area	Student notes
Long-term fix	
Owner	
Evidence	
Exception needed?	

Scenario B - Cluster-admin sprawl

Context: A review shows that multiple developer groups and a CI/CD service connection have broad cluster-admin rights across dev, test, and prod.

Student task: Design a least-privilege RBAC model using Microsoft Entra groups, namespace-scoped roles, and emergency access controls.

Decision area	Student notes
Highest risk	
Immediate action	
Long-term fix	
Owner	
Evidence	
Exception needed?	

Scenario C - Default network access

Context: A compromised frontend pod can reach database pods, internal APIs, and metadata endpoints because no network policies exist.

Student task: Create a staged microsegmentation plan with default deny, explicit app flows, DNS/logging egress, and testing gates.

Decision area	Student notes
Highest risk	
Immediate action	
Long-term fix	
Owner	

Decision area	Student notes
Evidence	
Exception needed?	

Scenario D - Defender sensor unhealthy

Context: The Containers plan is enabled, but runtime telemetry is missing and sensor pods are unhealthy on private AKS clusters.

Student task: Troubleshoot permissions, private connectivity, proxy/firewall, egress, Kubernetes version, and extension health.

Decision area	Student notes
Highest risk	
Immediate action	
Long-term fix	
Owner	
Evidence	
Exception needed?	

Scenario E - Production activation request

Context: A customer asks students to enable Defender for Containers in a live subscription during class.

Student task: List required approvals, scope boundaries, cost controls, rollback plan, named accounts, and evidence capture before proceeding.

Decision area	Student notes
Highest risk	
Immediate action	
Long-term fix	
Owner	
Evidence	
Exception needed?	

16. Knowledge check

Q1. Why is Defender for Containers not just an image scanner?

Answer: Because it also supports posture management, runtime telemetry, Kubernetes audit-log analysis, threat detection, and response integration.

Q2. What is the risk of assigning cluster-admin to application teams?

Answer: It gives broad ability to modify workloads, secrets, roles, and policies, increasing blast radius if the identity is compromised.

Q3. Why should vulnerable running images be prioritized over unused registry images?

Answer: Running images represent active exposure in the environment, especially if the workload is internet-facing or business-critical.

Q4. What does default-deny network policy achieve?

Answer: It blocks implicit pod-to-pod and pod-to-external communication until required paths are explicitly allowed.

Q5. Why use Microsoft Entra Workload ID?

Answer: It allows pods to access Azure resources without storing long-lived credentials or secrets in Kubernetes.

Q6. What should be captured when closing a vulnerability recommendation?

Answer: Image digest/version, package/fixed version, redeployment evidence, recommendation status, owner sign-off, and any exception record.

Q7. What are common blockers when enabling Defender for Containers?

Answer: Insufficient permissions, blocked egress/proxy restrictions, unsupported Kubernetes versions, missing AKS/registry resources, and unapproved costs.

Q8. What is the purpose of Defender XDR integration?

Answer: To correlate runtime/container alerts with broader incidents and support SecOps investigation and response workflows.

17. Glossary

Term	Meaning
AKS	Azure Kubernetes Service, Microsoft's managed Kubernetes service.
Admission control	Kubernetes mechanism that evaluates or mutates API

Term	Meaning
	requests before objects are persisted.
Azure RBAC	Azure role-based access control for Azure resource permissions.
ClusterRole / RoleBinding	Kubernetes RBAC objects used to grant permissions at cluster or namespace scope.
CSPM	Cloud Security Posture Management; identifies and prioritizes misconfigurations and exposure.
CWPP	Cloud Workload Protection Platform; protects workloads such as containers, servers, storage, databases, and AI services.
Defender sensor	Defender for Containers component deployed to collect runtime telemetry from Kubernetes nodes.
eBPF	Linux kernel technology used by security tools to observe runtime process/network activity efficiently.
Gated deployment	Admission-time security control that blocks or warns on risky deployments based on findings/rules.
Image digest	Immutable hash identifier for a container image. Prefer over mutable tags for high-assurance deployments.
Network policy	Kubernetes resource that allows or denies pod ingress and egress traffic.
Pod Security Admission	Kubernetes admission mechanism for applying pod security standards.
Secure Score	Defender for Cloud metric that helps prioritize security posture improvements.
Workload identity	Federated identity pattern that lets pods authenticate to Azure services without stored credentials.

18. References and further learning

The following references were used to align the student manual with current Microsoft documentation and the attached CESE323 module material. Students should consult the online documentation for the latest service-specific details and support matrices.

- [Microsoft Learn: Introduction to Microsoft Defender for Containers](#)
- [Microsoft Learn: Enable Defender for Containers in Microsoft Defender for Cloud](#)
- [Microsoft Learn: Defender for Containers deployment overview](#)
- [Microsoft Learn: Defender for Containers architecture](#)

- [Microsoft Learn: Container vulnerability assessment in Defender for Cloud](#)
- [Microsoft Learn: View and remediate vulnerabilities for running containers](#)
- [Microsoft Learn: Containers support matrix in Defender for Cloud](#)
- [Microsoft Learn: Enable gated deployment for Kubernetes clusters](#)
- [Microsoft Learn: AKS cluster security and upgrades best practices](#)
- [Microsoft Learn: AKS network policy best practices](#)
- [Microsoft Learn: AKS network connectivity and security best practices](#)
- [Microsoft Learn: Access and identity options for AKS](#)
- [Microsoft Learn: User roles and permissions in Defender for Cloud](#)
- [Microsoft Learn: Connect Azure subscriptions to Defender for Cloud](#)

Course-source references

- Attached slide deck: Defender for Containers - Hardening Kubernetes (AKS) and containerized workloads.
- CESE323 Cloud Security Envisioning & Strategy Workshop Syllabus.
- CESE323 Student Pre-Course Readiness Requirements.

Appendix A - Quick hardening checklist

Area	Check	Done
Scope	Subscription, resource group, AKS cluster, and registry are identified.	
Permissions	Students and operators have least-privilege roles.	
Plan	Containers plan is enabled only for approved scope.	
Components	Required Defender components are enabled and healthy.	
Inventory	AKS clusters, nodes, namespaces, workloads, registries, and images are visible.	
Identity	Microsoft Entra integration, Azure RBAC, Kubernetes RBAC, and workload identity are reviewed.	
Network	Network policies, private connectivity, ingress, WAF, and egress controls are reviewed.	
Pod security	Privileged/root containers and unnecessary capabilities are removed.	
Supply chain	Registry scanning, image patching, trusted registries, and gate policies are	

Area	Check	Done
	reviewed.	
Runtime	Alerts, audit logs, sensor telemetry, and Defender XDR visibility are validated.	
Operations	Owners, SLAs, evidence requirements, and exceptions are defined.	
Cleanup	Lab resources and paid plans are cleaned up or explicitly retained.	

Appendix B - Portal navigation quick reference

Goal	Portal path
Enable/review Defender plan	Azure portal > Microsoft Defender for Cloud > Environment settings > Subscription > Defender plans > Containers.
Review recommendations	Microsoft Defender for Cloud > Recommendations > filter by Kubernetes, AKS, containers, or registry.
View vulnerability findings	Microsoft Defender for Cloud > Recommendations > Vulnerabilities tab > Resource type: Container.
Review inventory	Microsoft Defender for Cloud > Inventory > filter for Kubernetes/containers.
Review alerts/incidents	Microsoft Defender portal / Defender XDR > Incidents & alerts.
Review permissions	Azure portal > Subscription or resource group > Access control (IAM).
Review AKS resources	Azure portal > Kubernetes services > target AKS cluster.